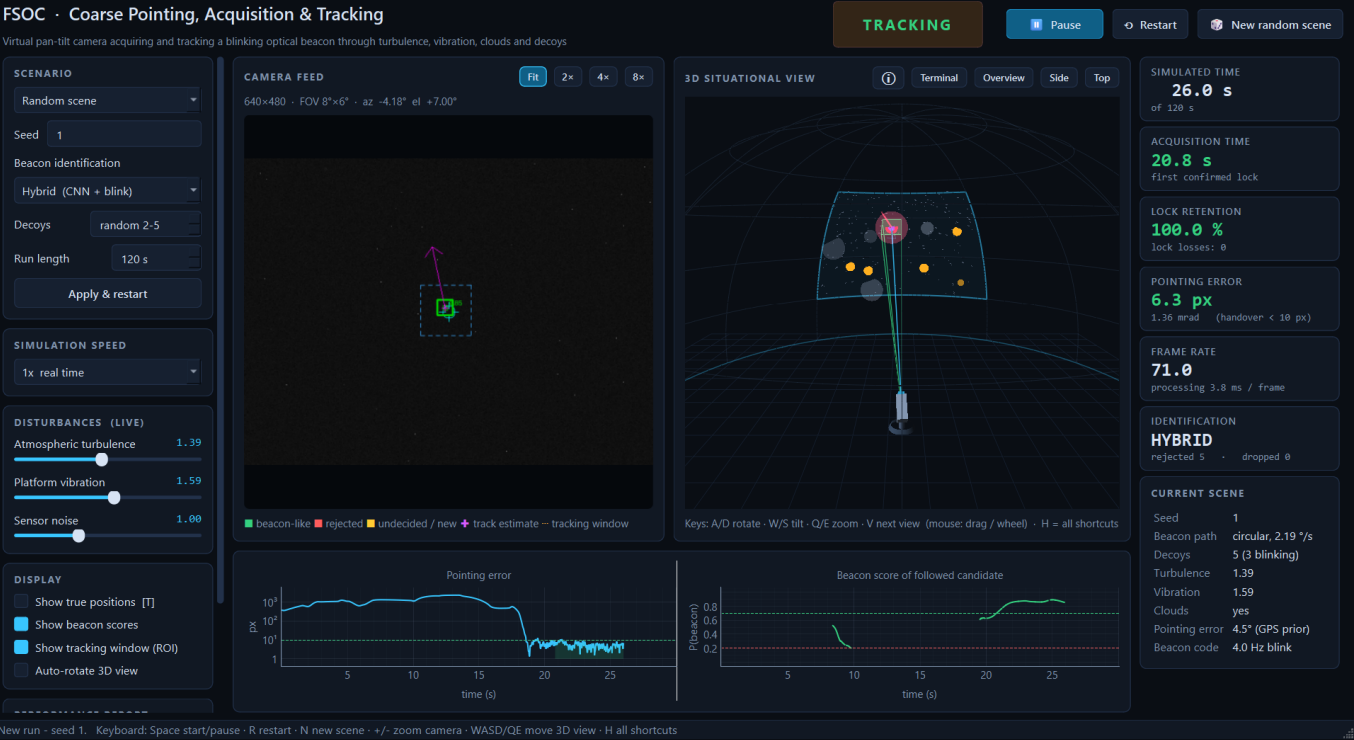


FSOC Coarse PAT Simulator

Virtual pan-tilt camera that **searches** the sky, **finds** a blinking optical beacon, **locks on** and **keeps it centred** — through turbulence, vibration, clouds and decoys.

FSOC · Coarse Pointing, Acquisition & Tracking

Virtual pan-tilt camera acquiring and tracking a blinking optical beacon through turbulence, vibration, clouds and decoys



CONTENTS

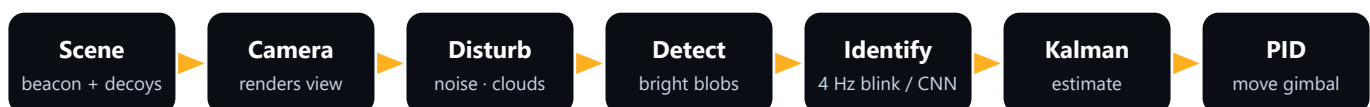
Inside this manual

Short text, many pictures. Each chapter is one page.

1	What it does	2
2	Install & launch	3
3	Screen tour	4
4	Run controls & tracker states	5
5	Setting up a run	6
6	Disturbances	7
7	Camera feed	8
8	Display options	9
9	3D situational view	10
10	Live plots & performance cards	11
11	Test scenarios	12
12	Beacon identification (AI)	13
13	Performance reports	14
14	Lightweight viewer & command line	15
15	Keyboard shortcuts	16
16	Advanced: config, evaluation, retraining	17
17	Troubleshooting & FAQ	18

1 • What it does

A ground terminal must point its laser at a moving partner (drone, aircraft, satellite). Before fine pointing can start, a wide camera must find the partner's **beacon** — a light that blinks at **4 Hz** — and hold it inside a 10-pixel **handover** circle. This app simulates exactly that stage.



One loop per frame (30 fps). Every box is a module in `fsoc/`.

CHAPTER 2

Install & launch

Pick one of three ways. Option A needs nothing installed.

A

Ready-made .exe (Windows)

Double-click `dist/FSOC_Simulator/FSOC_Simulator.exe`
Single-file variant: `dist/FSOC_Simulator.exe` (slower start)

B

Desktop app from source

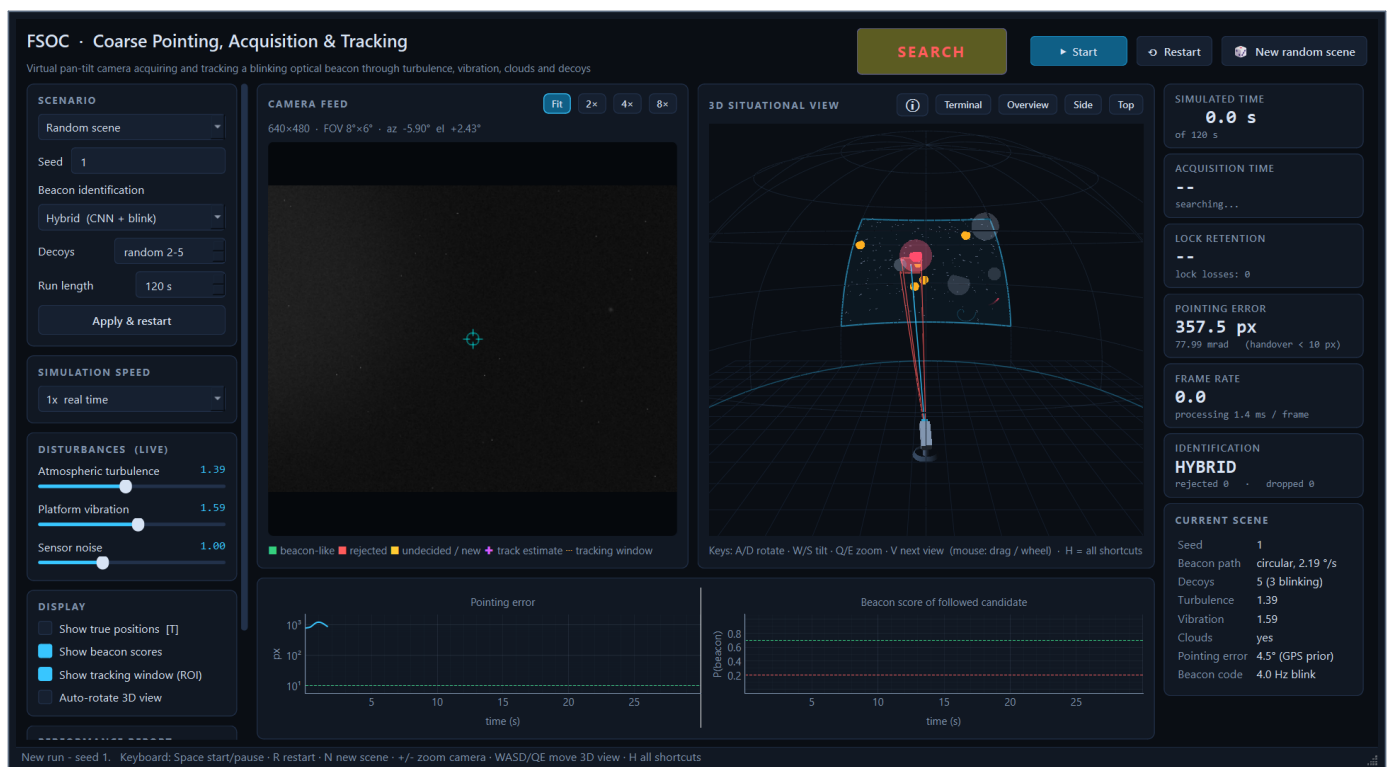
```
pip install -r requirements.txt
python app.py
```

C

Lightweight viewer / batch

```
python main.py or python main.py --headless --duration 120
```

Need	Details
Python	3.10 + (tested on 3.11)
Packages	numpy, opencv-python, PyYAML, PySide6, pyqtgraph, PyOpenGL (+ matplotlib, reportlab for PDF reports)
Screen	1366×768 or larger (layout adapts)
GPU	Any with OpenGL 2 (for the 3D view)



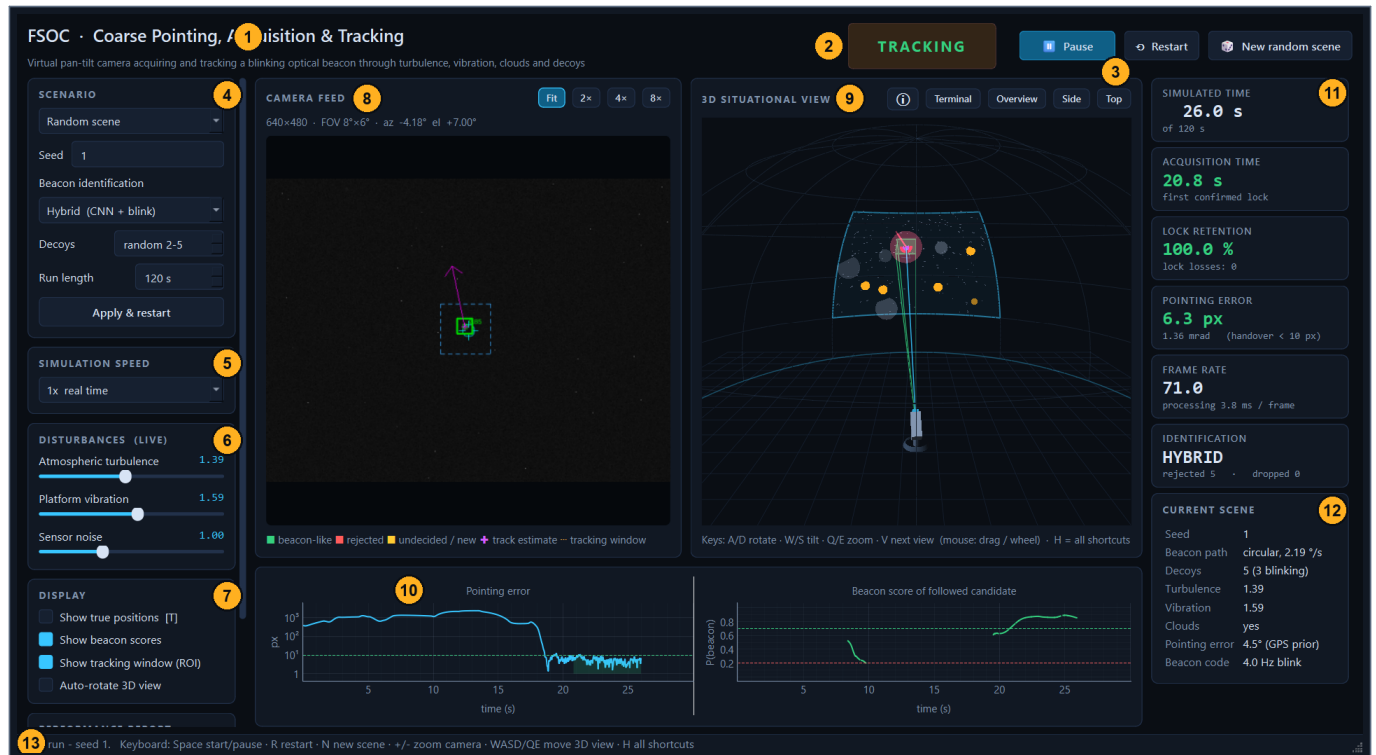
What you see after launch: window opens maximised, paused, state **SEARCH**. Press **Start** or **Space**.

TIP Check a packaged build without opening the window: `FSOC_Simulator.exe --selftest 20`, then read `runs/selftest.txt` (PASSED / FAILED).

CHAPTER 3

Screen tour

Everything on one screen. Numbers match the list below.



1	Title bar	App name and one-line description	8	Camera feed	What the camera sees + detections
2	State badge	SEARCH / ACQUIRING / TRACKING / COASTING	9	3D view	Terminal, sky dome, beacon, decoys, clouds
3	Run buttons	Start-Pause · Restart · New random scene	10	Live plots	Pointing error, beacon score
4	Scenario	Scene, seed, AI method, decoys, run length	11	Performance cards	Time, acquisition, retention, error, FPS, ID
5	Speed	0.5x ... Max	12	Current scene	Seed and what was randomised
6	Disturbances	Live sliders: turbulence, vibration, noise	13	Status bar	Messages, e.g. where a report was saved
7	Display	Overlays on/off, auto-rotate 3D			

CHAPTER 4

Run controls & tracker states

Three buttons drive a run; the badge tells you what the tracker is doing.

FSOC · Coarse Pointing, Acquisition & Tracking

Virtual pan-tilt camera acquiring and tracking a blinking optical beacon through turbulence, vibration, clouds and decoys

TRACKING

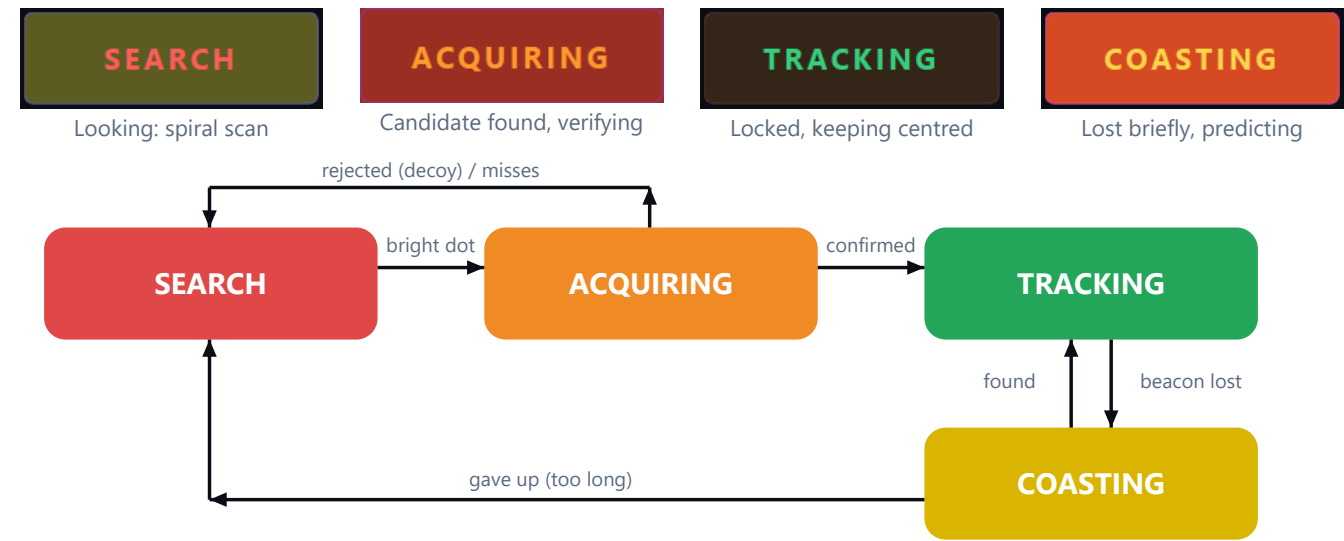
Pause

Restart

New random scene

Button	Does	Key
Start / Pause	Run or freeze the simulation	Space
Restart	Same scene again (same seed)	R
New random scene	New path, decoys, turbulence, seed	N

Tracker states



Short gaps (blink off, faint twinkle) do **not** leave TRACKING. Clouds cause COASTING; a rejected decoy sends the tracker back to SEARCH and is ignored for 4 s.

CHAPTER 5

Setting up a run

Left panel, top. Change options, then press **Apply & restart**.

SCENARIO

Random scene1

Seed12

Beacon identification

Hybrid (CNN + blink)3

Decoysrandom 2-54

Run length120 s5

Apply & restart6

- 1 **Scenario** — *Random scene* (new every run), *Default (fixed)*, or one of 6 test scenarios
- 2 **Seed** — blank = random. Type a number to replay an exact run
- 3 **Beacon identification** — Hybrid · CNN only · Blink test only · None (see ch. 12)
- 4 **Decoys** — *random 2-5* or fixed 2-8 (random scenes only)
- 5 **Run length** — 0-3600 s; 0 = *endless*. Default 120 s
- 6 **Apply & restart** — builds the new run

Simulation speed

Speed	Use for
0.5×	Slow motion — watch acquisition
1× real time	Default
2× / 4×	Faster runs
Max	As fast as the PC allows

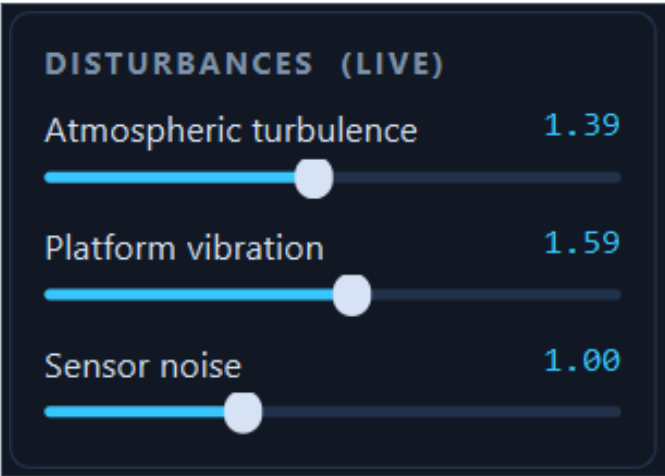
TIP Every report stores its **seed**. Copy it into the Seed box and press Apply & restart to see the exact same run again.

NOTE Decoy count only applies to *Random scene*. Fixed scenarios keep their own decoys.

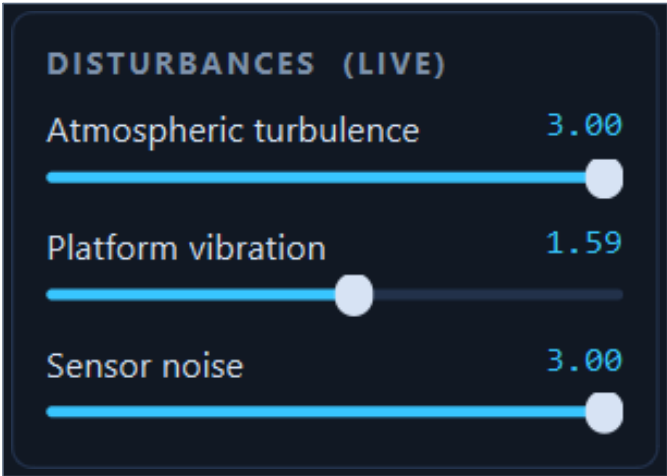
CHAPTER 6

Disturbances

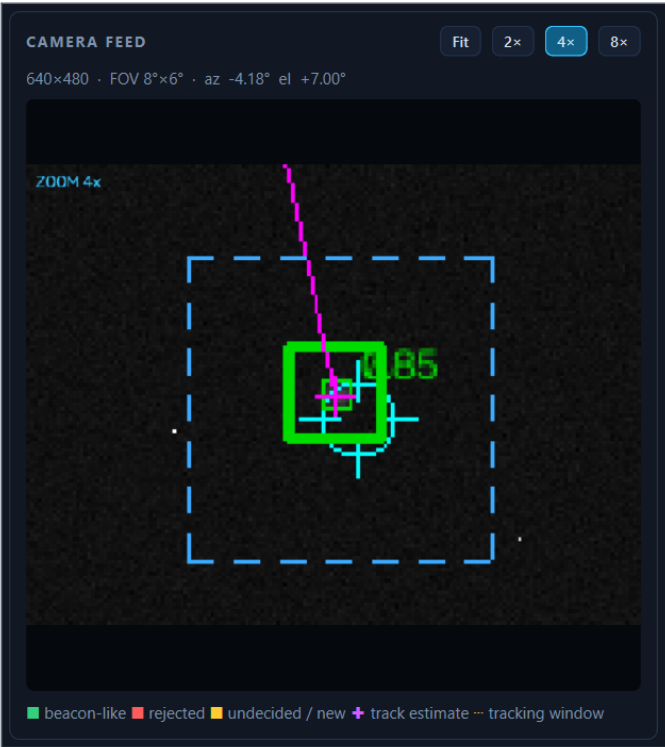
Three live sliders (0 = off, 1 = nominal, 3 = extreme). They act **instantly**, even while running.



Normal scene



Turbulence and noise pushed to 3.00



Result: clean



Result: grainy, beacon twinkles and wanders

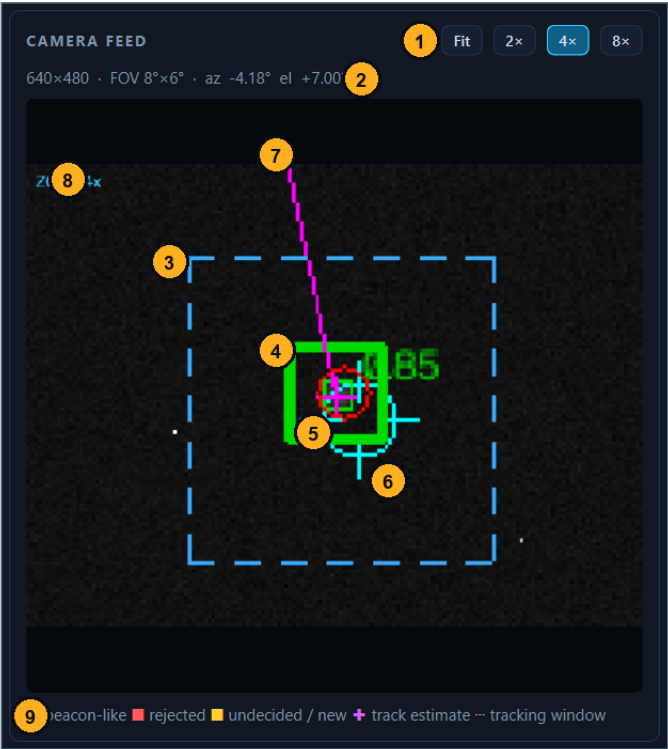
Slider	Simulates
Atmospheric turbulence	Twinkle, beam wander, blur, heat shimmer
Platform vibration	Gimbal jitter (7 Hz + 23 Hz tones + random)
Sensor noise	Shot / read noise, hot pixels, random pixel hits

Clouds, glints, dropouts and sun glare are set per scenario / in the config file (ch. 16).

CHAPTER 7

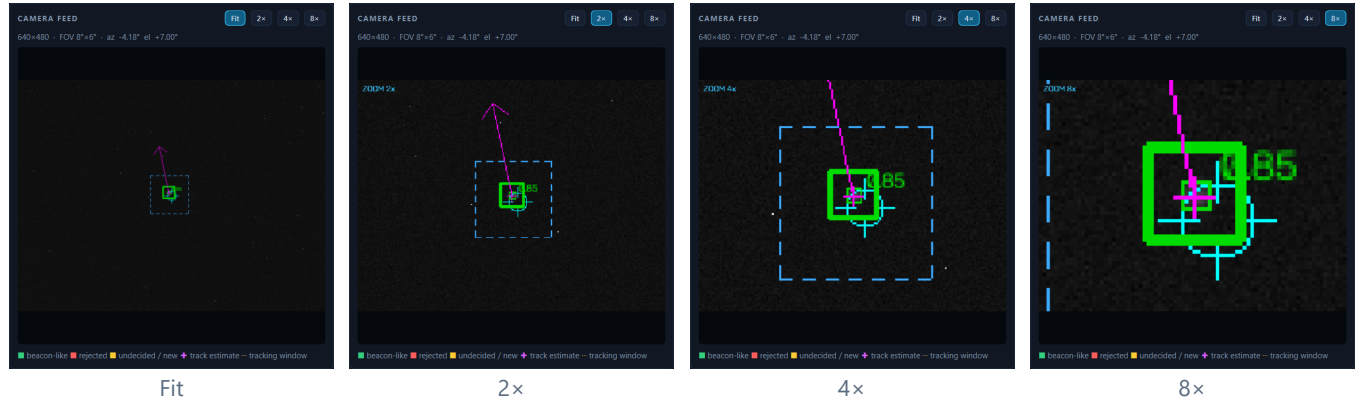
Camera feed

The 640×480 image the camera sees, with the tracker's view drawn on top. Shown here at 4× zoom with true positions on.



- 1 Zoom: Fit · 2× · 4× · 8× (also mouse wheel, + / - / 0)
- 2 Resolution, field of view, current azimuth / elevation
- 3 Tracking window (ROI) — a more sensitive search box
- 4 Candidate box + **P(beacon)** score. Green = beacon-like, red = rejected, yellow / ? = undecided
- 5 Red circle = **true** beacon (only with *Show true positions*)
- 6 Cyan crosshair + circle = boresight and 10 px handover zone
- 7 Magenta cross + arrow = Kalman estimate and where it will be in 0.5 s
- 8 Zoom label
- 9 Colour legend

Zoom levels (centred on the tracked beacon)



CHAPTER 8

Display options

Tick boxes change only what is drawn — never the tracking itself.

DISPLAY

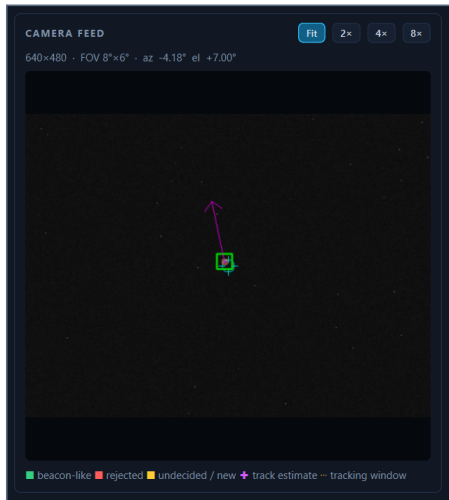
☐ Show true positions [T]

☒ Show beacon scores

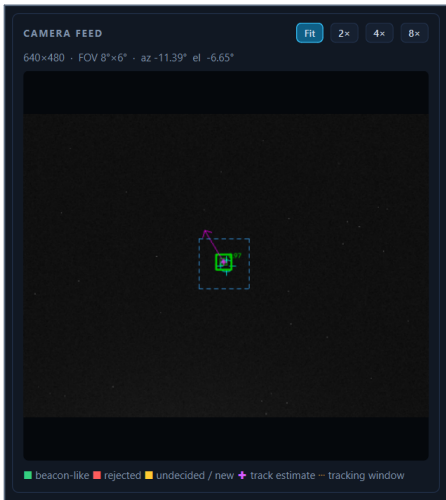
☒ Show tracking window (ROI)

☐ Auto-rotate 3D view

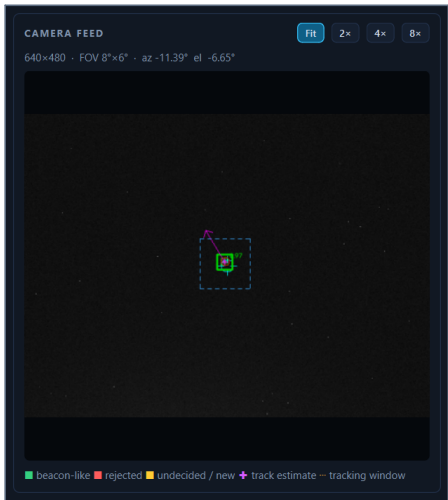
Show true positions [T]	Circles the real beacon (red) and decoys (orange)
Show beacon scores	P(beacon) number next to each candidate
Show tracking window	Dashed ROI box around the prediction
Auto-rotate 3D view	Slowly orbits the 3D scene



All overlays off



Scores + window on (default)



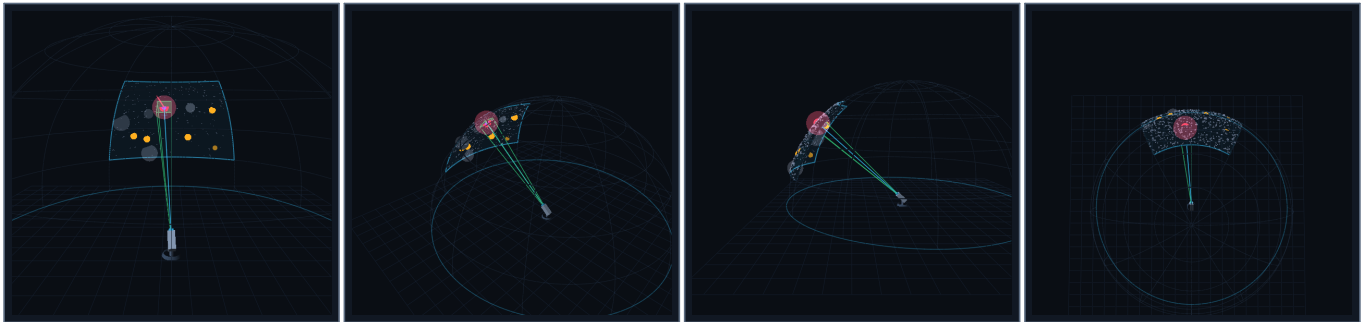
True positions on (decoy scenario)

TIP Use **T** to check the tracker: the green box should sit on the red truth circle.

CHAPTER 9

3D situational view

Sees the whole sky: where the camera points, where the beacon really is, decoys and clouds.



Terminal

Overview

Side

Top

LEGEND

- B Beacon**
true position of the target light; pulses with its 4 Hz blink; red trail = recent path
- Decoy**
other lights the tracker must ignore (steady or wrong blink rate)
- ▲ Camera (PAT terminal)**
pan-tilt camera on its mount; the head turns with the gimbal
- Camera field of view**
pyramid + footprint on the sky; colour = tracker state:
Search Acquiring Tracking Coasting
- Line of sight / search path**
where the camera points now, and where it has looked
- Tracker estimate**
where the Kalman filter believes the beacon is
- Cloud**
blocks or dims the beacon while it passes
- Simulated sky**
the region the beacon and decoys move in (az ±30°, el ±20°)

Moving the view

Mouse drag / wheel	Rotate / zoom
A / D	Rotate left / right
W / S	Tilt up / down
Q / E	Zoom in / out
V	Next preset view
Preset buttons	Terminal · Overview · Side · Top
(i) button or I	Open this legend

Field-of-view colour = tracker state



The cyan search path shows every place the camera has looked — the spiral scan becomes visible.

CHAPTER 10

Live plots & performance cards

Numbers update while the run is going.



1 Pointing error (px, log scale) — how far the camera is from the beacon

2 Dashed green line = 10 px handover limit

3 Green fill = periods with a confirmed lock

4 Beacon score of the candidate being followed

5 Accept threshold (0.7) — above: confirmed as beacon

6 Reject threshold — below: marked as decoy

SIMULATED TIME
26.0 s
of 120 s

ACQUISITION TIME
20.8 s
first confirmed lock

LOCK RETENTION
100.0 %
lock losses: 0

POINTING ERROR
6.3 px
1.36 mrad (handover < 10 px)

FRAME RATE
71.0
processing 3.8 ms / frame

IDENTIFICATION
HYBRID
rejected 5 · dropped 0

CURRENT SCENE
Seed 1
Beacon path circular, 2.19 °/s
Decoys 5 (3 blinking)
Turbulence 1.39
Vibration 1.59
Clouds yes
Pointing error 4.5° (GPS prior)
Beacon code 4.0 Hz blink

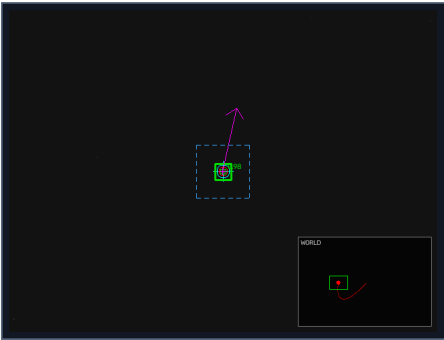
Performance cards

Simulated time	Elapsed / run length
Acquisition time	Seconds until the first confirmed lock
Lock retention	% of time locked after first lock; green ≥ 90, yellow ≥ 60, red below
Pointing error	px and mrad; green when inside handover
Frame rate	Frames per second and ms of processing per frame
Identification	Method, candidates rejected, false locks dropped
Current scene	Seed, beacon path & speed, decoys, turbulence, vibration, clouds, GPS error

CHAPTER 11

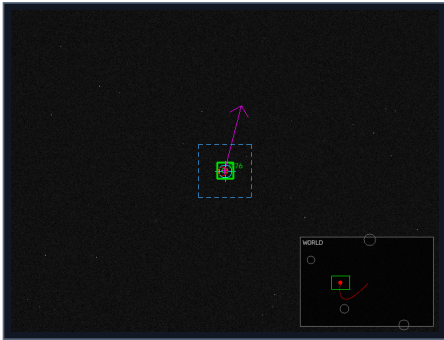
Test scenarios

Six fixed, repeatable test cases (seed 42). Choose them in the Scenario list. Pictures: camera at t = 20 s, truth on, world map bottom-right.



Clear sky

No disturbances — ideal baseline



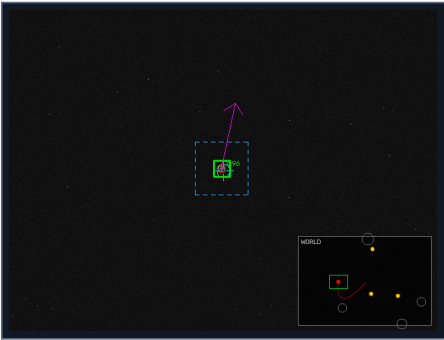
Strong turbulence

Heavy turbulence, vibration, sensor noise



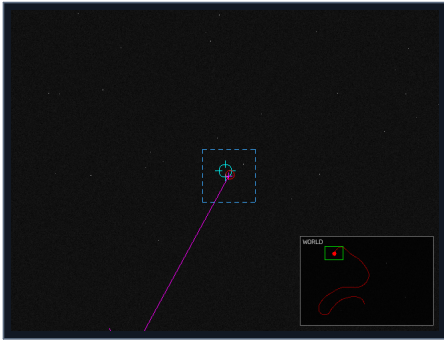
Occlusion

Dense clouds + dropouts: coasting & re-acquisition



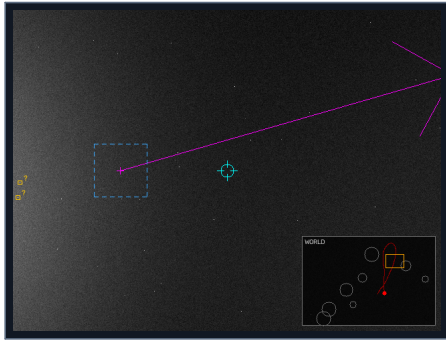
Decoys

Similar lights: pick the one blinking at 4 Hz



Fast target

Fast, manoeuvring beacon: agility



Worst case

Everything at once

Scenario	Acquired	Median acq. time	Time locked
01 Clear sky	100 %	7.4 s	88 %
02 Strong turbulence	100 %	11.4 s	52 %
03 Occlusion	100 %	27.5 s	32 %
04 Decoys	100 %	15.1 s	64 %
05 Fast target	100 %	20.3 s	63 %
06 Worst case	90 %	26.8 s	10 %

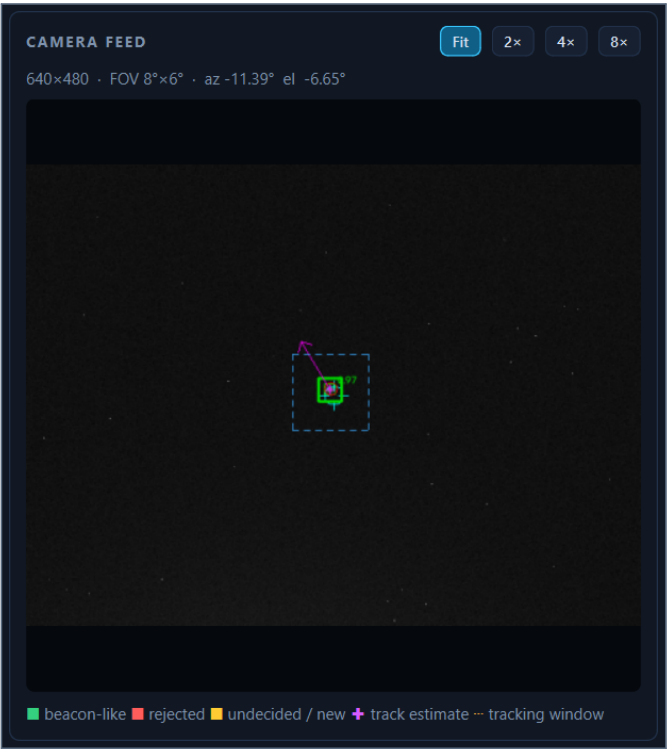
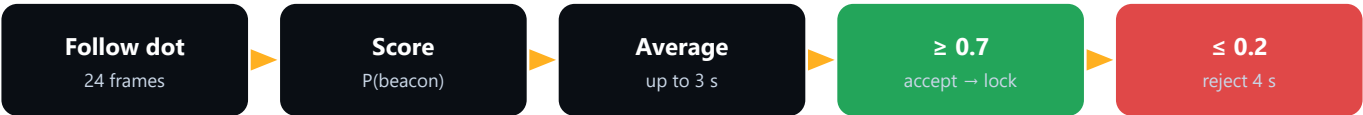
Measured over 10 seeds each (runs/evaluation/summary.md). Use them to compare settings.

CHAPTER 12

Beacon identification (AI)

In one frame every bright dot looks alike. The beacon is recognised by **how it blinks** over time.

Method	How it decides
Hybrid (default)	Average of CNN and blink test — fewest false locks
CNN only	BeaconNet: small neural net on the last 24 image patches (0.8 s)
Blink test only	Classical: strength of the 4 Hz peak in brightness
None	Baseline: lock onto the brightest dot (easily fooled)



Identification card: how many decoys were rejected and false locks dropped

Hybrid: green box on the true beacon (red circle); decoys (orange) ignored

Method (40 random scenes)	Acquired	Time locked	False-lock runs
Hybrid	92 %	72 %	8 %
None (brightest dot)	62 %	54 %	42 %

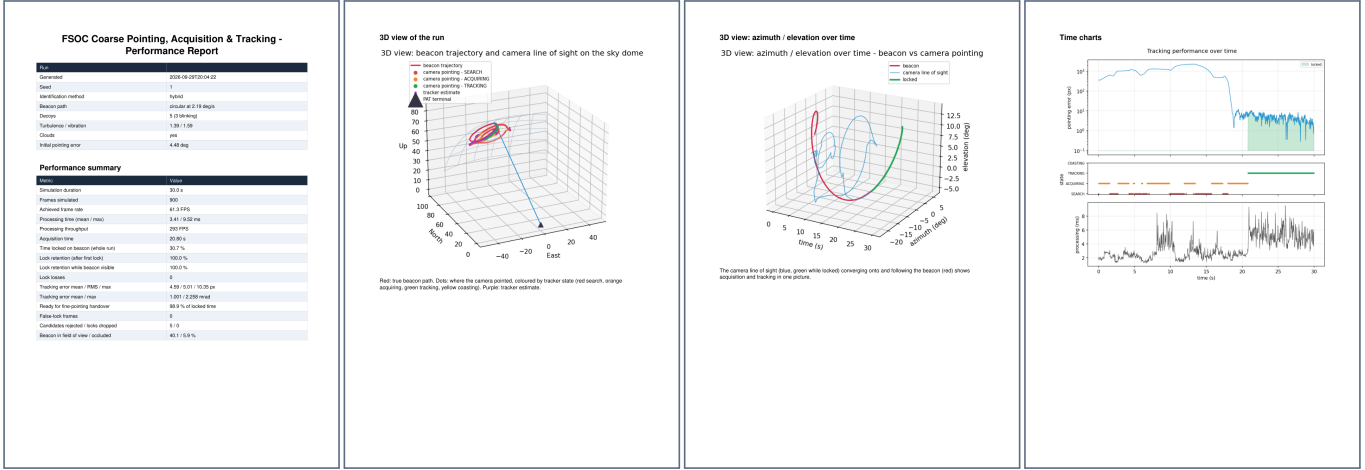
TIP Try it: choose 04 Decoys, set the method to None, Apply & restart, press T. Watch it chase the wrong light.

CHAPTER 13

Performance reports

Saved automatically when a run ends, or any time with **Save report now**.

File (runs/<run name>/)	Contents
report.html	Interactive report: tables + rotatable 3D views. Works offline
report.pdf	Same report, printable
frames.csv	One row per frame: state, positions, errors, timing
summary.json / .txt	All summary numbers



Run info + performance table

3D sky-dome view

Az / el / time 3D

Error, state timeline, CPU

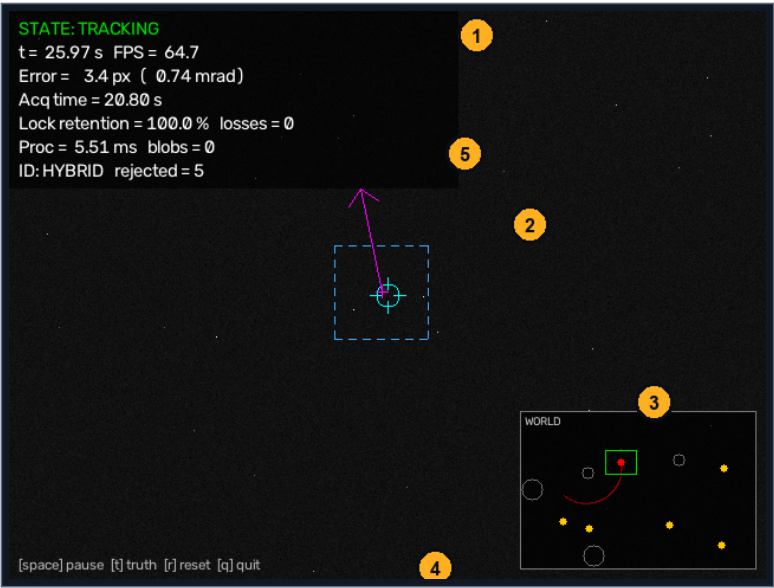
Key number	Meaning
Acquisition time	Time to first confirmed lock
Lock retention	% locked after first lock
Tracking error mean / RMS / max	Distance camera ↔ beacon (px, mrad)
Handover ready	% of locked time inside the 10 px circle
False lock frames	Frames locked on something that is not the beacon
Processing time / FPS	Speed of the pipeline

NOTE **Open reports folder** opens the runs folder in Explorer. With the .exe it sits next to the program.

CHAPTER 14

Lightweight viewer & command line

main.py — a small OpenCV window, or no window at all for fast batch runs.



- 1 HUD: state, time, FPS, error, acquisition, retention, ID
- 2 Candidate + estimate + tracking window
- 3 WORLD minimap: whole sky, camera box, beacon trail, decoys
- 4 Key hints
- 5 Velocity arrow

Command	Does
<code>python main.py</code>	Viewer, random scene
<code>python main.py --config scenarios/04_decoys.yaml</code>	Viewer, chosen scenario
<code>python main.py --seed 1</code>	Replay a seed
<code>python main.py --headless --duration 120</code>	No window, runs as fast as possible
<code>python main.py --name my_test</code>	Name the output folder runs/my_test
<code>python app.py --selftest 20</code>	Check the install / .exe, writes runs/selftest.txt

Viewer key	Action
Space	Pause
t	Show true positions
r	Reset
q / Esc	Quit (saves report)

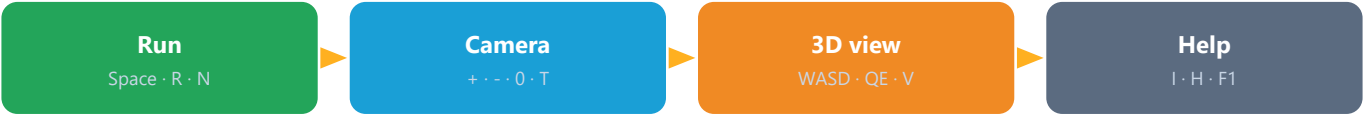
CHAPTER 15

Keyboard shortcuts

The whole app works without a mouse. Press **H** or **F1** in the app to see this list.

Space	Start / pause	R	Restart same scene
N	New random scene	T	Show / hide true positions
+	Camera zoom in	-	Camera zoom out
0	Camera fit	A	3D rotate left
D	3D rotate right	W	3D tilt up
S	3D tilt down	Q	3D zoom in
E	3D zoom out	V	Next 3D preset
I	3D legend	H / F1	Shortcut help

Groups at a glance



CHAPTER 16

Advanced: config, evaluation, retraining

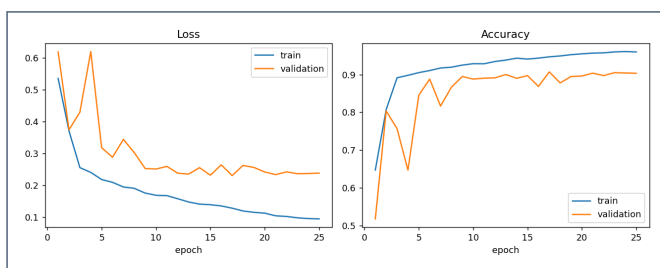
For experiments beyond the sliders.

File / command	Purpose
<code>config/default.yaml</code>	Every parameter, commented. Camera, detector, tracker, PID, each disturbance (enabled + strength)
<code>scenarios/*.yaml</code>	Only the values that differ from default. Copy one to make your own
<code>python tools/evaluate.py --suite random --seeds 1-40</code>	Many seeds → runs/evaluation/summary.md with 95 % CIs + box plots (resumes if stopped)
<code>python tools/compare_methods.py</code>	none / blink / cnn / hybrid side by side
<code>python tools/generate_dataset.py</code>	Make training clips for the CNN
<code>python tools/train_classifier.py</code>	Train → models/candidate/ (never overwrites the app's model)
<code>python tools/build_exe.py</code>	Build the .exe (see README)

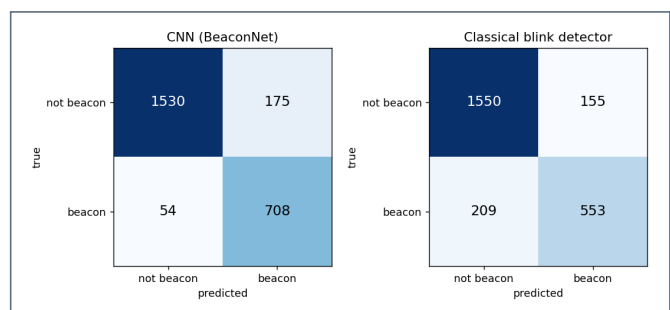
Example scenario file

```
# my_scenario.yaml - only list what changes
randomize: {enabled: false}
simulation: {seed: 42, duration: 90}
disturbances:
  turbulence: {strength: 2.0}
  dropouts: {enabled: true}
  glare: {enabled: true}
```

Run: `python main.py --config my_scenario.yaml`. Put it in `scenarios/` and it also appears in the app's list.



CNN training curves



CNN confusion matrix

CHAPTER 17

Troubleshooting & FAQ

Quick fixes.

Problem	Fix
3D view is black / app crashes at start	Update the graphics driver (needs OpenGL). Use <code>python main.py</code> as fallback
Stays in SEARCH for a long time	Normal for large GPS errors or fast beacons: the spiral widens. Lower turbulence or try another seed
Locked on the wrong light	Use Hybrid, not None. Check with T. Counts as a false lock in the report
Frame rate low	Speed = 1×, untick Auto-rotate, use a smaller window
Seed box says 'must be a whole number'	Type digits only; empty = random
Decoy setting ignored	Only works with Random scene; press Apply & restart
No report	Saved when the run ends. Endless runs: press Save report now
Single-file .exe opens slowly	It unpacks itself each time — use the folder version
Where are the reports?	runs/ in the project (or next to the .exe). Button: Open reports folder
Replay a run from a report	Copy the seed from the report → Seed box → same scenario & method → Apply & restart

Glossary

PAT	Pointing, Acquisition and Tracking
FSOC	Free-Space Optical Communication
Beacon	Light on the partner terminal, blinking at 4 Hz
Decoy	Any other light (aircraft, star, glint)
Boresight	Centre of the camera image
Handover	Beacon within 10 px of centre → fine pointing can take over
Seed	Number that fixes all randomness of a run
mrاد	Milliradian; 1 px $\approx$ 0.22 mrad